

Migration Plan: nano-vllm Attention to cuTile/TileGym

This document outlines the step-by-step migration plan to integrate **cuTile / TileGym** custom attention kernels into `nano-vllm`. By replacing the Linux-only dependencies (**FlashAttention** and **Triton**) with **cuTile** and pure PyTorch operations, we enable **GPU-accelerated native execution directly on Windows OS** (via PowerShell) while maintaining the original PyTorch/FlashAttention codebase for reference and comparison.

1. Architectural Highlights & Experience Leveraged

This plan builds directly on the optimization findings from our previous tasks:

- **Phase 1 (FMHA):** Proven best tile configurations (64×64 and 128×64), memory coalescing, and prefetch pipelining.
- **Phase 2 (LLM-from-Scratch):**
 - **Causal Padding:** If sequence length $T < 64$, pad to 64 using `F.pad` before running the custom cuTile kernel, and slice back to T to avoid falling back to PyTorch SDPA.
 - **No Cache Clearing (`fill_`):** Remove unnecessary `.fill_(0.0)` cache resets since causal masking (`offs_m >= offs_n`) automatically ignores future slots.
 - **Direct Prefill Writing:** Write projections directly into pre-allocated cache buffers on the GPU during the prefill forward pass rather than performing post-hoc copies.
 - **CUDA Graph Harmony:** Bypassing CPU-GPU synchronization by loading dynamic inputs (like step indices) from 1-element GPU tensors.

2. Phase-by-Phase Migration Plan

🔧 Phase 1: Triton to Pure PyTorch KV Store Migration

Currently, `nano-vllm` uses a Triton kernel (`store_kvcache_kernel`) to copy projected K/V tensors into the physical cache blocks. Triton is Linux-only, preventing native execution on Windows.

- **Action:** Replace the Triton-based `store_kvcache` function in `nanovllm/layers/attention.py` with a pure PyTorch flat index-write operation:

```
def store_kvcache(key: torch.Tensor, value: torch.Tensor, k_cache: torch.Tensor, v_cache: torch.Tensor,
                  slot_mapping: torch.Tensor):
    D = key.shape[1] * key.shape[2]  # num_heads * head_dim
    N = key.shape[0]

    # Flat views of KV cache and projections
    k_cache_flat = k_cache.view(-1, D)
    v_cache_flat = v_cache.view(-1, D)
    key_flat = key.view(-1, D)
    value_flat = value.view(-1, D)

    # Write only active slots (ignoring slots that map to -1)
    mask = slot_mapping != -1
    valid_slots = slot_mapping[mask].long()

    k_cache_flat[valid_slots] = key_flat[mask]
    v_cache_flat[valid_slots] = value_flat[mask]
```

- **Validation:** Test natively on Windows/PowerShell CPU simulation or GPU to ensure values are copied into identical memory locations.

Phase 2: cuTile Custom Kernels for Prefill and Paged Decode

We will write two custom cuTile kernels to replace `flash-attn` calls:

1. Prefill Attention (`cutile_fmha`):

- Computes causal self-attention for variable-length prompts.
- **Padding Mechanism:** If the prompt length $T < 64$, we pad Q, K, V to 64 tokens, launch our optimized cuTile kernel, and slice the output back to T to avoid any SDPA fallback.

2. Paged Decode Attention (`cutile_fmha_paged_decode`):

- Performs PagedAttention decoding using `block_table`.
- **Block Resolution:** Compute physical cache addresses directly on the GPU within the cuTile load instructions: $\text{block_id} = \text{block_table}[\text{batch_idx}, \text{col_idx} // \text{block_size}]$
 $\text{slot_offset} = \text{col_idx} \% \text{block_size}$

Phase 3: Integration into `nanovllm/layers/attention.py`

To preserve the original PyTorch/FlashAttention source code, we will introduce a `use_cutile` flag in the config/context. The `Attention` layer's `forward` pass will conditionally route execution:

```
class Attention(nn.Module):
    # ...
    def forward(self, q: torch.Tensor, k: torch.Tensor, v: torch.Tensor):
        context = get_context()
        k_cache, v_cache = self.k_cache, self.v_cache

        # 1. Update Cache
        if k_cache.numel() and v_cache.numel():
            store_kvcache(k, v, k_cache, v_cache, context.slot_mapping)

        # 2. Select Backend Routing
        if context.use_cutile:
            if context.is_prefill:
                # Launch custom cuTile prefill attention (with causal padding if T < 64)
                o = cutile_fmha_prefill(q, k, v, causal=True, ...)
            else:
                # Launch custom cuTile PagedAttention decode
                o = cutile_fmha_paged_decode(q, k_cache, v_cache, context.block_tables, ...)
        else:
            # ORIGINAL PyTorch / FlashAttention code paths
            if context.is_prefill:
                o = flash_attn_varlen_func(q, k, v, ...)
            else:
                o = flash_attn_with_kvcache(q.unsqueeze(1), k_cache, v_cache, ...)
        return o
```

Phase 4: CUDA Graph Capture Harmony

- Ensure the custom cuTile PagedAttention kernels do not trigger any CPU-GPU synchronization (e.g. by passing the current step/token indices as 1-element GPU tensors).
- The JIT-compiled cuTile kernels will be automatically recorded during `ModelRunner.capture_cudagraph()` for each bucketed batch size.

3. Step-by-Step Windows Native Setup & Execution (with cuTile)

Once migrated, the original Linux dependencies are eliminated. You can run the GPU-enabled server directly in PowerShell:

1. Install CUDA-compatible PyTorch & cuTile:

```
pip3 install torch --index-url https://download.pytorch.org/whl/cu121
pip install tilegym[tileiras]
```

2. Download Model:

```
python src/download_model.py --repo Qwen/Qwen2.5-0.5B-Instruct --dest ~/huggingface
mv ~/huggingface/Qwen2.5-0.5B-Instruct ~/huggingface/Qwen3-0.6B
```

3. Execute Benchmark:

```
python bench.py --use-cutile
```

Summary of Modifications

1. Windows Native Path Expansion: Updated download_model.py using os.path.expanduser so that the directory paths containing ~ resolve correctly on Windows PowerShell.
2. Syntax Error Fixes: Fixed broken multiline print syntax errors in test_layers.py and run_cpu_sim.py.
3. No Process Group Requirement for Unit Tests: Mocked torch.distributed in test_layers.py when running outside of rank-based contexts, allowing simple single-process unit testing.
4. cuTile Kernels & Wrapper: Created cutile_attention.py containing: • A variable-sequence-length causal FMHA prefill kernel using padded 4D tensors (avoiding SDPA fallbacks by padding to a multiple of 64). • A dynamic PagedAttention decoding kernel resolving block tables internally on the GPU. • Robust try-except imports for cuda.tile to prevent startup crashes on CPU-only machines.
5. Conditional Routing in Main Attention: Updated attention.py to: • Lazily and conditionally import triton and flash_attn . • Fall back to the pure PyTorch flat-view index write (store_kvcache_pytorch) on Windows or when cuTile is used. • Route to cutile_fmha_prefill or cutile_fmha_paged_decode if use_cutile=True or the NANO_VLLM_USE_CUTILE=1 environment variable is present.
6. Command-Line Integration in Benchmark: Updated bench.py to parse --use-cutile using argparse and set the environment variable. ———

Step-by-Step TDD Verification Guide for Your RTX 5070 Machine

Run the following commands in order in your PowerShell terminal:

Step 1: Run CPU Simulation & Existing Unit Tests

These check that existing linear/rotary layers and the vLLM scheduler logic work perfectly on CPU without requiring any GPU or CUDA packages:

```
# Run the core scheduler and sequence model simulation
python src/cpu_sim/run_cpu_sim.py

# Run layer-specific unit tests (Linear & RoPE)
python src/tests/test_layers.py
```

Step 2: Run the CPU/GPU Verification Suite

Verify the logic of our migrated KV-store, causal padding, and paged cache lookup:

```
python src/tests/test_migration.py
```

Step 3: Set Up the GPU Environment (RTX 5070)

Installs Windows-native CUDA-compatible PyTorch and the cuTile compiler package:

```
# Install GPU PyTorch (ensure your CUDA toolkit matches the wheel version)
pip install torch --index-url https://download.pytorch.org/whl/cu121

# Install TileGym/cuTile compiler library
pip install tilegym[tileiras]
```

Step 4: Download and Prepare the Target Model

Download the model to the local cache directory:

```
# Download Qwen2.5-0.5B-Instruct to your user profile directory
python src/download_model.py --repo Qwen/Qwen2.5-0.5B-Instruct --dest ~/huggingface

# Rename folder to match path expected by example and benchmark
Move-Item -Path ~/huggingface/Qwen2.5-0.5B-Instruct -Destination ~/huggingface/Qwen3-0.6B
```

Step 5: Execute and Validate Output

Run the example text generation using cuTile:

```
# Run using the environment variable flag to route to cuTile
$env:NANO_VLLM_USE_CUTILE="1"
python example.py
```

Step 6: Benchmark Throughput

Measure tokens/sec with and without the cuTile backend:

```
# 1. Run baseline FlashAttention/original path (will raise exception on Windows if flash-attn)
python bench.py

# 2. Run cuTile accelerated path
python bench.py --use-cutile
```

I have aligned the implementation of our custom cuTile kernels in `cutile_attention.py` with the reference designs from NVIDIA/TileGym/src/tilegym/ops .

Specifically, we leveraged:

1. Online Softmax Scaling: Computes and normalizes scaling using intermediate maximum log-sum-exp variables (`m_i` , `l_i`) in float32 for maximum numerical precision, matching the reference FMHA tiling strategy.
2. Dynamic Block Transposition: Loaded keys transposed to (`head_dim`, `block_size`) directly at the hardware load level via custom transposition orders (`order=(0, 3, 2, 1)`), avoiding high-overhead global memory transpose operations.
3. Internal Block Table Lookup: Resolved physical memory addresses within the CUDA execution loop:

```
blockW_id = blockW_table [batchW_idx,j]
```

and fetched from `k_cache[physical_block_id, offset, head, :]` dynamically.